如何构建一个更“好”的知识库？

原创 架构部 大淘宝技术 2026年6月10日 00:32 浙江 37人

本文深入探讨构建高质量 RAG 知识库的垂直技术原理与工程实践。文章首先界定知识库作为外部记忆系统的角色，并引入 RAGAS 框架从检索相关性、生成忠实度及答案相关性维度建立评估标准。随后详细拆解离线索引与在线查询流程，重点分析文档切分策略如 Late Chunking 和意图驱动切分，对比稀疏、稠密及混合检索范式，并阐述HyDE等查询增强技术。此外，文章探讨 Cross-Encoder 重排序机制以优化精度，介绍 AutoRAG 自动化优化、 QuIM-RAG 问题倒排索引及 OpenViking 文件系统范式等前沿架构，旨在通过系统性技术选型解决幻觉、召回不准等问题，实现知识库性能的端到端优化。

01 什么是 Agent 构建中的知识库？什么时候应该用它？

考古一下，RAG 的起源

RAG（Retrieval-Augmented Generation，检索增强生成） 由 Facebook AI Research（现 Meta AI）于 2020 年首次提出。

项目	内容
论文	*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*
作者	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin 等
发表	NeurIPS 2020
链接	https://arxiv.org/abs/2005.11401

论文的核心贡献在于提出了一种将**参数化记忆（Parametric Memory）** 与**非参数化记忆（Non-Parametric Memory）** 相结合的架构：

- **参数化记忆：**预训练 seq2seq 模型（如 BART）的模型权重
- **非参数化记忆：**Wikipedia 语料的密集向量索引，通过 DPR（Dense Passage Retriever）构建

这一架构在开放域问答（Open-Domain QA）任务上显著超越了纯参数化模型，奠定了后续 RAG 研究的基础。

知识库的定义

在 Agent 构建的语境下，**知识库（Knowledge Base）** 是一个外部记忆系统，用于存储和检索不在模型参数中的信息。它作为 RAG 架构的核心组件，承担非参数化记忆的角色。

RAG 的基本工作流程：

Query → Retriever（检索器） → Top-K Documents → Context Augmentation → Generator（生成器） → Response

其中，知识库的核心接口，就是上传和召回。不同版本和理论，就是召回的内容和排序的区别。

使用知识库可以解决什么问题?

这个问题应该回到 LLM 的固有局限上，知识库是一种对应的解决方案：

局限性	技术描述	知识库解决方案
Knowledge Cutoff	模型参数在训练时冻结，无法获取训练后的信息	外部知识库支持实时更新，无需重新训练模型
Private Data Access	预训练语料不包含企业内部数据	将私有数据索引化后接入检索系统
Hallucination	模型可能生成与事实不符的内容	基于检索到的真实文档生成，提供 grounding
Context Window Limit	即使 100K+ context，仍无法容纳大规模语料库	按需检索，仅将相关片段注入上下文
Attribution	纯生成模型难以提供答案来源	可追溯至具体文档和段落

适用场景分析

结合前面几点，使用场景也比较清晰了。

适合构建知识库的场景：

场景类型	典型应用
企业知识问答	内部文档检索、客服系统、产品手册查询
垂直领域	法律法规检索、医疗文献查询、金融研报分析
知识管理	个人笔记检索、学术文献管理
动态数据	需要访问实时更新内容的场景

不需要知识库的场景：

场景类型	原因
通用知识问答	模型参数已包含足够的世界知识
创意生成	不依赖外部事实，无需检索
推理任务	依赖模型推理能力而非外部知识
小规模数据	数据量可直接放入 prompt context

RAG vs Long Context

随着上下文窗口的扩展（Claude 200K, Gemini 1M+），需要重新审视 RAG 的适用边界：

维度	Long Context	RAG
数据规模	适合 <100K tokens	支持任意规模语料库
成本	每次请求消耗全量 tokens	仅检索结果计入 context，成本可控
数据更新	需要重新拼接完整 context	增量索引，更新高效
召回精度	依赖模型 in-context 注意力机制	独立检索系统，召回可调优
延迟	TTFT 随 context 长度线性增长	检索 + 生成，延迟相对稳定
可解释性	难以定位信息来源	可追溯到具体检索结果

选型建议：

- 数据量 < 50K tokens 且更新频率低 → Long Context
- 数据量大、更新频繁、需要精确召回 → RAG
- 混合方案：RAG 粗筛 + Long Context 精读



如何评判一个知识库的好坏？

如题，我们想要构建更好的知识库，那么首先需要定义"好"的标准。

评估框架：RAGAS

RAGAS（Retrieval Augmented Generation Assessment） 是目前最广泛采用的 RAG 评估框架，其核心价值在于**无参考评估（Reference-Free）**——无需人工标注 ground truth 即可进行自动化评估。

项目	内容
论文	*RAGAS: Automated Evaluation of Retrieval Augmented Generation*
链接	https://arxiv.org/abs/2309.15217
发表	2023

RAGAS 将 RAG 系统的评估分解为三个核心维度：

维度	评估对象	核心问题
Context Relevance	检索模块	是否召回了相关且聚焦的上下文？
Faithfulness	生成模块	是否忠实地利用了检索到的上下文？
Answer Relevance	生成模块	生成的答案是否直接且完整地回应了问题？

这三个维度相互独立但互补，共同覆盖 RAG 系统的端到端性能。RAGAS 的关键洞察：**RAG 系统的失败往往是检索和生成环节共同造成的**，因此必须分别评估，才能定位问题根因。下面来看下这几个环节可以对应的指标有些什么。

检索质量指标

检索环节的目标是：**召回与 query 相关的文档片段，并将相关内容排在前面**。

Context Precision（上下文精确率）

定义：评估检索器将相关文档排在不相关文档之上的能力。

计算方法：

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K} (\text{Precision@k} \times v_k)}{\text{Top-K 中相关项的总数}}$$

其中：

- K 为检索的 chunks 总数
- $v_k \in \{0, 1\}$ 为排名 k 处的相关性指示器
- $\text{Precision@k} = \frac{\text{true positives@k}}{\text{true positives@k+false positives@k}}$

直观理解：如果检索了 5 个 chunks，相关的 2 个排在第 1、2 位，比排在第 4、5 位的 precision 更高。

Context Recall（上下文召回率）

定义：评估回答问题所需的信息有多少被成功检索到。

计算方法：

$$\text{Context Recall} = \frac{\text{参考答案中被检索上下文支持的声明数}}{\text{参考答案中的总声明数}}$$

具体步骤：

1. 将参考答案分解为多个 claims（声明）
2. 判断每个 claim 是否可归因于检索到的上下文
3. 计算被支持的 claims 占比

注意：Context Recall 需要参考答案（reference），因此不是完全的 reference-free 指标。

传统 IR 指标：（除 RAGAS 定义的指标外，传统 IR（Information Retrieval）指标仍然适用）

指标	定义	关注点
Precision@K	Top-K 结果中相关文档的占比	结果纯净度
Recall@K	所有相关文档中被检索到 Top-K 的比例	召回覆盖度
F1@K	Precision@K 和 Recall@K 的调和平均	精确率与召回率的平衡
MRR	第一个相关文档排名倒数的均值	首个正确结果的位置
NDCG@K	考虑相关性分级的排序质量	整体排序质量
Hit@K	Top-K 中是否包含至少一个相关文档	二元召回判断

Precision、Recall 与 F1 的关系：

$$\text{Precision@K} = \frac{|\text{Top-K} \cap \text{相关文档}|}{K}$$

$$\text{Recall@K} = \frac{|\text{Top-K} \cap \text{相关文档}|}{|\text{所有相关文档}|}$$

$$\text{F1@K} = \frac{2 \times \text{Precision@K} \times \text{Recall@K}}{\text{Precision@K} + \text{Recall@K}}$$

F1 是 Precision 和 Recall 的调和平均数，用于在两者之间取得平衡。当 Precision 和 Recall 差异较大时，F1 会偏向较小的那个值，因此 F1 高意味着两者都不能太低。

生成质量指标

生成环节的目标是：**基于检索到的上下文，生成准确、相关的答案**。

Faithfulness（忠实度）

定义：生成的答案在事实上是否与检索到的上下文一致。取值范围 [0, 1]，值越高表示答案越忠实于上下文。

计算方法：

$$\text{Faithfulness} = \frac{|\text{被上下文支持的声明}|}{|\text{答案中的总声明数}|}$$

具体步骤：

1. 使用 LLM 从答案中提取所有声明（claims）

2. 对每个声明，验证是否能从检索上下文中推断

3. 计算被支持的声明占比

示例：

问题	Einstein 在哪里、什么时候出生?
上下文	Albert Einstein (born 14 March 1879) was a German-born theoretical physicist...
低忠实度答案	Einstein 出生于德国，1879 年 3 月 20 日
分析	声明 1 "出生于德国" ✓ 支持；声明 2 "3 月 20 日" ✗ 不支持
得分	1/2 = 0.5

Answer Relevance（答案相关性）

定义：答案是否直接且恰当地回应了问题。该指标不考虑事实准确性，而是惩罚不完整或包含冗余信息的答案。

计算方法：

$$\text{Answer Relevance} = \frac{1}{N}\sum_{i=1}^{N}\cos(E_{g_i}, E_o)$$

其中：

- E_o 为原始问题的 embedding
- E_{g_i} 为基于答案生成的第 i 个问题的 embedding
- N 为生成的问题数量（默认 3）

核心思想：如果答案正确回应了问题，那么从答案反向生成的问题应该与原问题高度相似。

幻觉问题的深入分析

Faithfulness 指标的本质是检测幻觉。RAG 系统的幻觉可进一步细分为三类：

幻觉类型	描述	根因
Contextual Hallucination	模型被不相关的检索内容误导	检索质量问题
Knowledge Conflict	检索内容与模型参数知识矛盾时，模型选择了错误来源	模型决策问题
Source Attribution Failure	无法正确引用信息来源，或引用不存在的来源	归因机制缺失

参考论文：https://arxiv.org/abs/2601.19927

幻觉检测方法

方法	类型	特点
LLM-as-a-Judge	通用	使用 GPT-4 等大模型判断
Lynx	开源	幻觉检测专用模型
HHEM	商用	轻量级检测器

参考论文：https://arxiv.org/abs/2503.21157
不同 RAG 应用场景（医疗、法律、通用 QA）对检测器的要求不同，需根据具体场景选择。

RAG 场景的特殊考量

传统 IR 指标基于语义相似度评估检索质量，但在 RAG 场景下存在一个核心问题：**语义相似 ≠ 对 LLM 有用**。

ICLERB（In-Context Learning Embedding and Reranker Benchmark）提出了端到端评估思路，这意味着：一个"好"的检索结果，不仅要语义相关，还要能有效支撑 LLM 生成正确答案。

`检索候选文档 → 注入 LLM 生成答案 → 评估答案准确性 → 反推检索器效果`

参考论文：https://arxiv.org/abs/2411.18947
以上，在理解了评估标准后，接下来拆解知识库构建的完整流程，分析每个环节的优化空间。

03 构建知识库分为几步？

知识库的构建可以分为两个阶段：**离线索引阶段（Indexing）** 和 **在线查询阶段（Querying）**。本章节结合idealab平台（https://idealab.alibaba-inc.com/#/aistudio）的操作进行讲解。

离线索引阶段：`Load → Split → Embed → Store`

步骤	名称	说明	示例
1	Load（加载）	从各种来源提取原始数据	PDF/Word/HTML/MD
2	Split（切分）	将长文档切分为片段	固定长度/语义切分
3	Embed（向量化）	将文本块转换为向量	Embedding 模型编码
4	Store（存储）	存入向量数据库并建立索引	FAISS/Milvus/OpenSearch

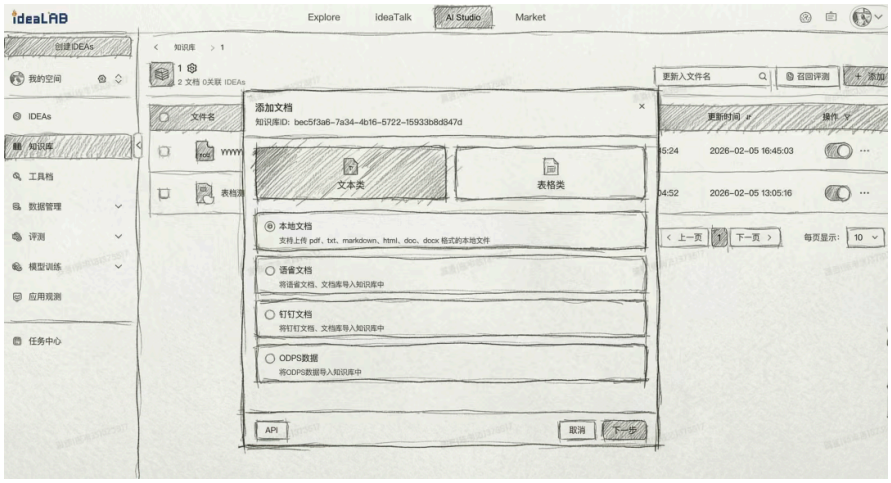
在线查询阶段：`Query → Retrieve → Rerank → Generate`

步骤	名称	说明	示例
5	Query（查询处理）	对用户查询进行预处理和增强	查询改写/意图识别
6	Retrieve（检索）	从向量库召回相关文档	向量相似度/Top-K 召回
7	Rerank（重排序）	对初筛结果精排	Cross-Encoder 精排
8	Generate（生成）	将上下文送入 LLM 生成答案	上下文增强/答案生成

离线索引阶段

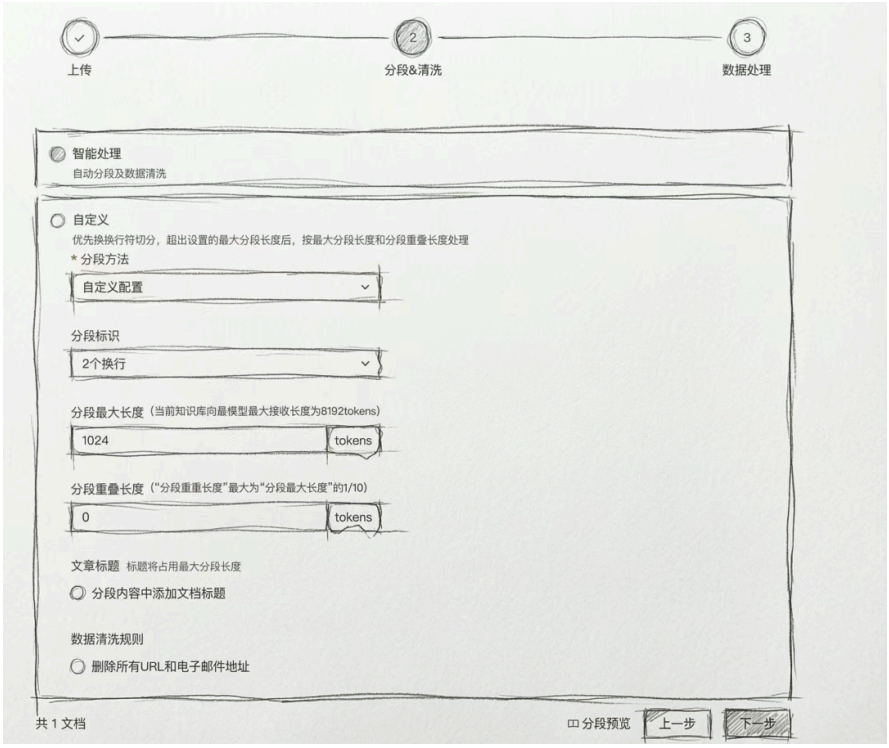
- Step 1: Load（文档加载）

这一步很好理解，就是将原始数据从各种来源和格式中提取出来。 目前idealab提供的知识库支持的有odps、语雀、钉钉文档、本地文件。



- Step 2: Split（文档切分）

将长文档切分为适合检索和上下文注入的片段（chunks）。这是影响检索质量的**关键环节**。
目前idealab提供的知识库支持的有默认智能切分（使用Opensearch切分方案），自定义切分（固定长度，符号切分），自定义工具切分。



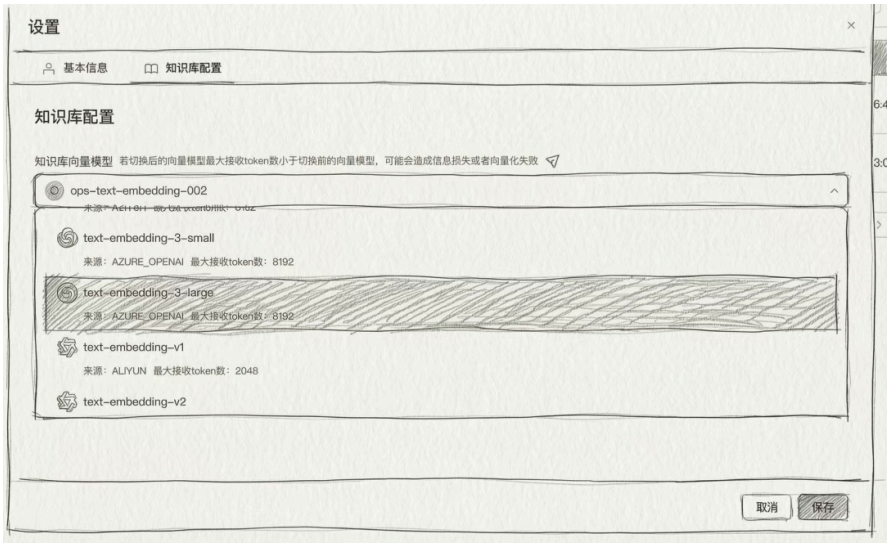
切分策略	原理	优劣势
固定长度切分	按 token/字符数等分	简单高效，但可能切断语义
指定符号切分	按句号，换行符等符号进去切分	兼顾语义边界
语义切分	基于 embedding 相似度断句	语义完整，但计算成本高
Late Chunking	先编码完整文档，再切分取均值	保留全局上下文，效果好
意图驱动切分	预测用户查询，动态规划边界	最优但复杂度高

关键参数：

- `chunk_size`：块大小，通常 256-1024 tokens
- `chunk_overlap`：重叠区域，通常 10%-20%，防止切断关键信息

- Step 3: Embed（向量化）

使用 Embedding 模型将文本块转换为稠密向量。本环节idealab提供了多种模型可供选择。



- Step 4: Store（存储与索引）

将向量及其元数据存入向量数据库，建立高效检索索引。

在线查询阶段

- Step 5: Query（查询处理）

对用户原始查询进行预处理和增强。这一步需要Agent的搭建者进行处理，最为简单的方式就是交给大模型自己来。充分信任基模的能力。 以下是一些常见的手段。

查询优化技术	说明	效果提升
Query Rewriting	重写为更适合检索的形式	语义对齐
Query Expansion	扩展同义词、相关概念	提升召回
HyDE	先生成假设答案，再用答案检索	语义桥接
Multi-Query	生成多个查询变体，合并结果	覆盖度提升

- Step 6: Retrieve（向量检索）

从向量数据库中召回与查询最相关的文档片段。

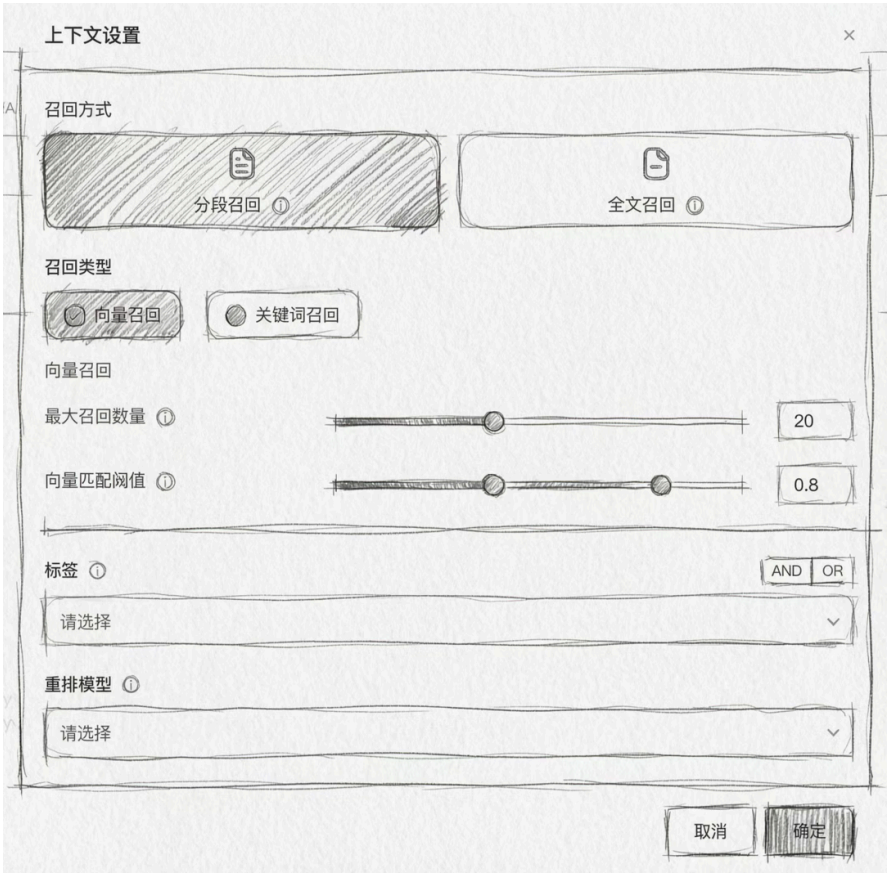


Query Embedding → ANN Search → Top-K Chunks

检索模式：

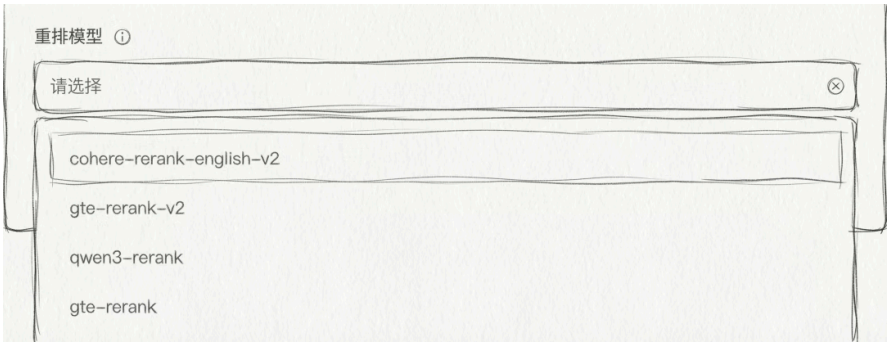
- **稠密检索：**基于向量相似度（余弦、内积）
- **稀疏检索：**基于词频统计（BM25）或学习权重（SPLADE）
- **混合检索：**稠密 + 稀疏，取长补短

同样的，idealab支持多种配置项；



- Step 7: Rerank（重排序，可选）

对初筛结果进行精排，解决初步召回不够准确的问题,尤其是混合召回后的排序。提升最终送入 LLM 的内容质量。



- Step 8: Generate（答案生成）

将检索到的上下文与用户问题一起送入 LLM 生成最终答案。

```
Prompt = System Prompt + Retrieved Contexts + User Query
                                ↓
                               LLM
                                ↓
                        Grounded Answer
```

知识库的开源项目和案例

上述是 RAG 的常用实践路径，以及 idealab 提供了搭建 Agent 知识库的能力。但作为扩展 LLM 能力的一个方案，其选型和能做的还有很多想象空间。不妨站的更高一点看看别人搞了些啥。

- 案例一：AutoRAG – 自动化 RAG 模块优化框架

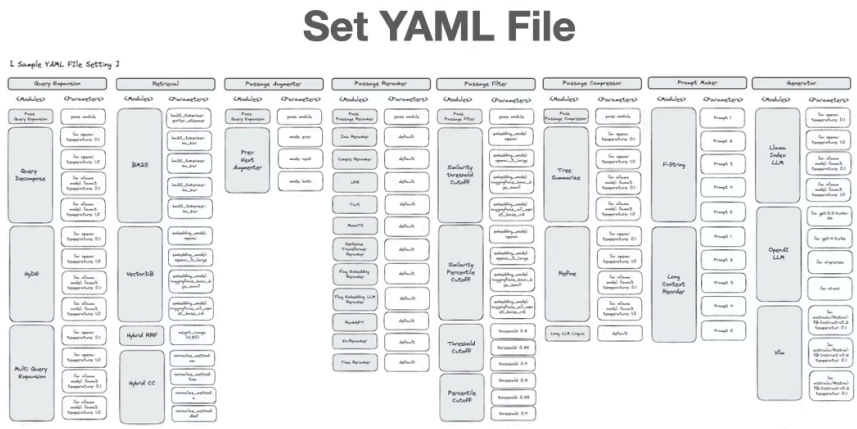
项目信息	内容
论文	*AutoRAG: Automated Framework for optimization of Retrieval Augmented Generation Pipeline*
链接	https://arxiv.org/abs/2410.20878
GitHub	https://github.com/Marker-Inc-Korea/AutoRAG

解决的问题：

RAG 系统涉及众多模块（分块策略、Embedding 模型、检索方式、Reranker 等），不同模块组合在不同数据集上表现差异很大。手动调优耗时且难以找到最优解。

核心方法：

AutoRAG 提供自动化的 RAG Pipeline 优化框架：



用户数据集 → AutoRAG 自动探索 → 最优模块组合

↓

测试多种 RAG 模块配置：

- Chunking: 固定/语义/递归
- Embedding: BGE/E5/OpenAI...
- Retrieval: Dense/Sparse/Hybrid
- Reranker: Cross-Encoder/ColBERT...

↓

基于评估指标自动选择最佳组合

适用场景：

- 需要为特定领域数据集优化 RAG 配置
- 缺乏调优经验或资源
- 希望系统化比较不同方案

- 案例二：QuIM-RAG – 问题倒排索引匹配

项目信息	内容
论文	*QuIM-RAG: Advancing Retrieval-Augmented Generation with Inverted Question Matching for Enhanced QA Performance*
发表	IEEE Access, vol. 12, pp. 185401-185410, 2024
链接	https://arxiv.org/abs/2501.02702

应用背景：

部署在一个**日访问量数千次的高流量网站**，用于回答复杂问题。语料库包含 500+ 页的领域文档。

解决的问题：

传统 RAG 在处理大量数据时存在**信息稀释**和**幻觉**问题——直接用 query 检索文档片段，语义匹配不够精准。

核心创新 - 问题倒排索引：

```
传统 RAG:
  User Query → 直接匹配 Document Chunks → 召回

QuIM-RAG:
  离线阶段: Document Chunks → LLM 生成潜在问题 → 问题索引
  在线阶段: User Query → 匹配生成的问题 → 定位最相关 Chunks
```

将"Query-Document 匹配"转化为"Query-Query 匹配"，提升检索精度。

评测结果：

- 使用 BERT-Score 和 RAGAS 指标评估
- 在两项指标上均优于传统 RAG 架构

- 案例三：OpenViking – 文件系统范式的上下文数据库

项目信息	内容
GitHub	https://github.com/volcengine/OpenViking
作者	字节跳动火山引擎

解决的问题：

传统 RAG 存在以下痛点：

- **上下文碎片化**：记忆在代码中、资源在向量库、技能分散各处
- **扁平化存储**：缺乏全局视角，难以理解信息的完整上下文
- **检索黑盒**：隐式检索链出错时难以调试
- **记忆迭代有限**：只是被动记录交互，缺少与任务相关的主动记忆

核心创新 - 文件系统范式（Filesystem Paradigm）：

```
传统 RAG:
  文档 → 切分 → 向量化 → 扁平存储 → 语义检索

OpenViking:
  上下文 → 文件系统组织 → 分层加载 → 目录递归检索
                ↓
    统一管理 memories / resources / skills
```

技术特点：

特性	说明
分层上下文加载	L0/L1/L2 三层结构，按需加载，节省 Token 成本
目录递归检索	目录定位 + 语义搜索，精准获取上下文
可视化检索轨迹	检索过程可观测，便于调试优化
自动会话管理	自动提取长期记忆，Agent 越用越智能

与传统 RAG 的对比：

维度	传统 RAG	OpenViking
存储结构	扁平化向量存储	层次化文件系统
检索方式	纯语义搜索	目录定位 + 递归搜索
可观测性	检索链黑盒	可视化轨迹
记忆能力	被动记录	主动提取与进化

适用场景：长时运行的 Agent、需要复杂上下文管理的场景、对检索可解释性要求高的应用。

小结：

知识库构建没有标准答案，需要根据数据特点和业务场景选择合适的架构模式。核心原则：

1. **分块粒度**要匹配信息单元的自然边界
2. **Embedding 模型**要匹配语料语种和领域
3. **检索策略**要兼顾语义召回和精确匹配
4. **架构模式**要根据查询复杂度选择（线性/条件/分支/循环）

04 如何获得更好的切分（Chunking）？

切分（Chunking）是知识库构建中**影响最大但最容易被忽视**的环节。切分质量直接决定了：

- 检索能否召回完整的答案信息

- 上下文是否包含足够的语义
- 是否会引入无关噪声

切分的核心挑战

切分存在一个**两难困境**：

Chunk 太大	Chunk 太小
✓ 语义完整，上下文丰富	✓ 检索精准，噪声少
✗ 可能包含无关信息	✗ 可能丢失必要上下文
✗ 占用更多 context window	✗ 语义不完整，难以理解

核心目标：在粒度和完整性之间找到平衡点。

常见切分策略对比

- 固定长度切分（Fixed-size Chunking）

最简单的方法：按固定 token 数切分，通常加上重叠区域。

```
# 伪代码示例
chunk_size = 512        # 每块 512 tokens
chunk_overlap = 50      # 重叠 50 tokens
```

优势	劣势
实现简单，速度快	可能在句子/段落中间切断
结果可预测	不考虑语义边界
无额外计算成本	上下文可能不完整

适用场景：快速原型、对切分质量要求不高的场景

- 递归切分（Recursive Chunking）

按层次结构递归切分：先尝试按段落分，段落太长则按句子分，句子太长则按字符分。

```
优先级：段落(\n\n) → 换行(\n) → 句号(。.) → 逗号(,,) → 字符
```

优势	劣势
尊重自然语义边界	块大小不均匀
保持段落/句子完整性	可能产生极短的块
LangChain 默认推荐方法	对格式不规范的文档效果差

适用场景：通用文档处理，结构化文本

- 语义切分（Semantic Chunking）

基于语义相似度判断切分边界：相邻句子语义差异大时切分。

```
句子1 ←相似→ 句子2 ←相似→ 句子3 ←不相似→ 句子4
        [      Chunk 1      ]      [  Chunk 2  ]
```

计算方法：

1. 对每个句子计算 embedding
2. 计算相邻句子的余弦相似度
3. 相似度低于阈值处切分

优势	劣势
语义完整性好	计算成本高（需要对每句 embedding）
切分边界更合理	效果依赖 embedding 模型质量

关于成本效益的研究：

论文 *"Is Semantic Chunking Worth the Computational Cost?"*（https://arxiv.org/abs/2410.13070）的研究发现：

> 语义切分的计算成本与性能提升不成正比

在文档检索、证据检索、答案生成三个任务上的实验表明，语义切分相比固定长度切分，性能提升有限，但计算成本显著增加。

结论：简单分块 + 合理重叠可能是更实用的选择。

进阶切分策略

- Late Chunking（延迟切分）

项目	内容
论文	*Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models*
链接	https://arxiv.org/abs/2409.04701
作者	Jina AI, 2024

核心问题：

传统方法"先切分，再编码"会导致每个 chunk 丢失来自其他 chunks 的上下文信息。

```
传统方法 (Early Chunking):
  文档 → 切分为 Chunks → 分别 Embedding → 存储
                ↓
        每个 Chunk 独立编码，丢失上下文

Late Chunking:
  文档 → Transformer 编码所有 Tokens → 切分 → Mean Pooling → 存储
                ↓
        Chunk 的 Embedding 保留了全局上下文
```

为什么有效：

Transformer 的注意力机制使每个 token 都"看到"了整个文档。先编码再切分，每个 chunk 的表示中已经融入了全局上下文。

使用条件：

- 需要支持长上下文的 Embedding 模型（如 Jina Embeddings v2 8K）
- 文档长度不超过模型的上下文窗口

效果：在各类检索任务上优于传统切分方法，无需额外训练。

- 意图驱动动态切分（Intent-Driven Dynamic Chunking）

项目	内容
论文	*Intent-Driven Dynamic Chunking: Segmenting Documents to Reflect Predicted Information Needs*
链接	https://arxiv.org/abs/2602.14784

核心思想：

切分边界应该由"用户可能问什么问题"来决定，而非文档本身的结构。

```
传统切分：根据文档结构/长度切分
        ↓
问题：切分边界可能切断答案

意图驱动切分：
   1. 用 LLM 预测文档可能被问到的问题
   2. 根据问题对应的答案范围，动态规划最优切分边界
   3. 确保每个 chunk 包含完整的"可回答单元"
```

效果：

- 检索准确率提升 5%-67%
- 分块数量减少 40%-60%
- 答案覆盖率保持 93%-100%

适用场景：对检索质量要求高的场景，特别是长文档和异构文档。

- Small-to-Big 策略

检索时使用小粒度 chunk，生成时扩展为大粒度上下文。

```
索引阶段：
  文档 → 细粒度切分（句子级） → Embedding → 存储
            同时记录每个句子的父级段落

检索阶段：
  Query → 检索最相关的句子 → 扩展为其所在的段落/章节 → 送入 LLM
```

优势	实现方式
检索精准（小 chunk）	LlamaIndex: `SentenceWindowNodeParser`
上下文充分（大 context）	LangChain: `ParentDocumentRetriever`

切分参数选择

- Chunk Size

场景	推荐大小	理由
通用问答	256-512 tokens	平衡精度和上下文
精确检索	128-256 tokens	更小的块，更精准的匹配
长文档摘要	512-1024 tokens	保留更多上下文
代码检索	按函数/类切分	保持代码逻辑完整

- Chunk Overlap

```
Chunk 1: [____________________]
Chunk 2:             [____________________]
                     ↑
                重叠区域 (Overlap)
```

重叠比例	效果
0%	无重叠，可能丢失边界信息
10-20%	推荐值，平衡覆盖和冗余
>30%	冗余过多，增加存储和检索成本

切分策略选型建议

决策问题	条件	推荐策略
Q1: 文档是否有清晰的结构?	是（有章节/标题）	基于文档结构切分
	否	进入 Q2
Q2: 是否有长上下文 Embedding 模型?	是 + 文档 < 8K tokens	Late Chunking
	否	进入 Q3
Q3: 对检索精度要求有多高?	极高	Small-to-Big 策略
	高	递归切分 + 合理重叠
	一般	固定长度切分 + 10-20% 重叠
Q4: 是否愿意投入计算成本?	是	可尝试意图驱动切分
	否	坚持简单方法

实践建议

1. **从简单开始：** 先用递归切分 + 10% 重叠建立 baseline
2. **评估驱动：** 用 RAGAS 的 Context Precision/Recall 评估切分效果
3. **关注边界：** 检查切分是否切断了关键信息（答案、实体）
4. **元数据保留：** 切分时保留来源、章节、页码等元数据
5. **迭代优化：** 根据 bad case 分析调整切分策略



如何获得更好的召回（Retrieval）？

召回（Retrieval）是 RAG 系统的核心环节。检索质量直接决定了 LLM 能否获得正确的上下文信息——如果召回阶段就漏掉了相关文档，后续的 Rerank 和生成都无法弥补。

检索范式概述

当前主流的检索方法可分为三类：

检索范式	原理	代表方法	优势
稀疏检索	基于词项匹配	BM25、SPLADE	精确匹配、可解释性强、计算高效
稠密检索	基于 Embedding 向量相似度	E5、BGE、GTE 等 Bi-Encoder	语义理解能力强、跨语言支持
混合检索	结合稀疏和稠密检索	多路召回 + 融合排序（RRF）	取长补短，覆盖度与精度兼顾

稀疏检索（Sparse Retrieval）

- BM25

BM25（Best Matching 25）是经典的稀疏检索算法，基于词频统计进行相关性打分：

$$\text{BM25}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

其中：

- $f(q_i, D)$ 为查询词 q_i 在文档 D 中的词频
- $|D|$ 为文档长度，avgdl 为平均文档长度
- k_1 和 b 为可调参数（通常 $k_1 = 1.2$，$b = 0.75$）

优势	劣势
无需训练，开箱即用	无法理解语义（同义词、上下位词）
计算高效，适合大规模	对查询和文档的词汇必须精确匹配
对专业术语、实体名称匹配准确	长尾查询效果差

稠密检索（Dense Retrieval）

稠密检索使用 Embedding 模型将文本映射为稠密向量，通过向量相似度（余弦、内积）进行检索。

- Embedding 模型选型

模型	参数量	上下文	多语言	特点
BGE-M3	0.6B	8192	✓	多语言、多粒度、多任务统一；支持稀疏+稠密混合
E5-large-v2	0.3B	512	✗	英文 SOTA，多阶段训练
GTE-large	0.3B	8192	✓	长文本处理能力强
Jina Embeddings v3	0.6B	8192	✓	支持 Late Chunking
Qwen3-Embedding	0.6B-8B	8192	✓	MTEB 多项 SOTA

混合检索（Hybrid Retrieval）

混合检索结合稀疏和稠密方法的优势：

混合检索架构流程：

阶段	处理步骤	说明
1. 输入	User Query	用户查询
2. 并行召回	稀疏检索（BM25）→ Top-K_1	精确匹配
	稠密检索（Dense）→ Top-K_2	语义理解
3. 融合排序	RRF / 加权线性组合	合并两路结果
4. 输出	最终 Top-N	送入 Reranker 或 LLM

- 融合策略

1. Reciprocal Rank Fusion (RRF)

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

其中 $\text{rank}_r(d)$ 是文档 d 在第 r 路召回中的排名，k 通常取 60。

2. 加权线性组合

$$\text{Score}(d) = \alpha \cdot \text{Score}_{\text{sparse}}(d) + (1 - \alpha) \cdot \text{Score}_{\text{dense}}(d)$$

α 可通过验证集调优，通常在 0.3-0.7 之间。

- 混合检索的优势

场景	稀疏检索表现	稠密检索表现	混合效果
专业术语查询	✓ 精确匹配	✗ 可能语义漂移	最优
同义词/语义查询	✗ 无法匹配	✓ 语义理解	最优
实体名称查询	✓ 精确匹配	△ 取决于训练	最优
长尾/复杂查询	✗ 覆盖不足	✓ 泛化能力强	最优

进阶检索策略

- 层次化检索（Hierarchical Retrieval）

项目	内容
论文	*Dense Hierarchical Retrieval for Open-Domain Question Answering*
链接	https://arxiv.org/abs/2110.15439

当语料库规模庞大时，直接段落级检索可能导致上下文丢失。层次化检索采用两阶段策略：

阶段	输入	处理	输出
Stage 1: 文档级检索	Query	检索相关文档	Top-M 文档
Stage 2: 段落级检索	Query + Top-M 文档	在相关文档内检索段落	Top-K 段落
融合	两阶段结果	用文档级相关性校准段落排名	最终排序

优势：

- 避免短段落丢失全局上下文
- 利用文档标题、章节结构等层次信息
- In-Doc 和 In-Sec 负采样策略提升训练效果

- 查询增强（Query Enhancement）

用户的原始查询往往不够清晰或完整，查询增强技术可以显著提升召回效果。

1. 查询改写（Query Rewriting）

使用 LLM 将用户查询改写为更适合检索的形式：

```
原始查询：RAG 怎么评估
改写后：如何评估 RAG 系统的检索质量和生成质量？有哪些评估指标？
```

2. HyDE（Hypothetical Document Embeddings）

项目	内容
论文	*Precise Zero-Shot Dense Retrieval without Relevance Labels*
链接	https://arxiv.org/abs/2212.10496

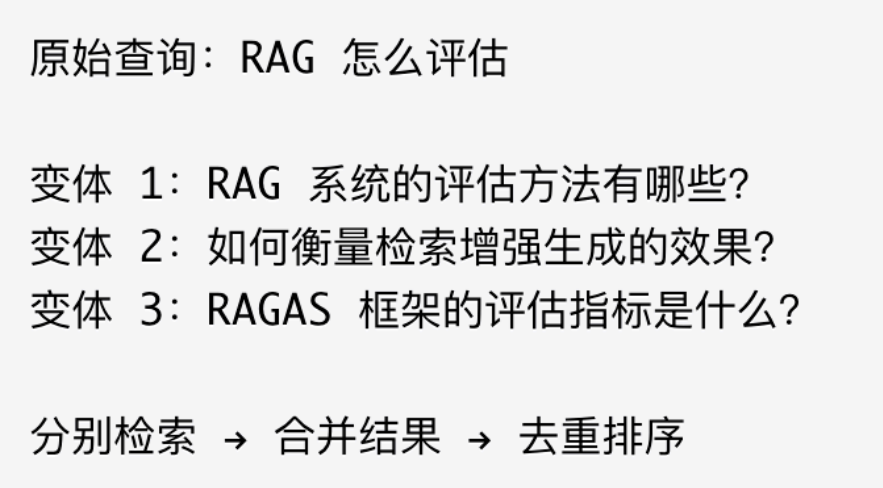
核心思想：先用 LLM 生成假设性答案，再用答案的 Embedding 进行检索。

```
Query: "RAG 的评估指标有哪些？"
          ↓
LLM 生成假设答案: "RAG 系统的评估指标包括 Faithfulness、Answer Relevance、
                   Context Precision、Context Recall 等..."
          ↓
用假设答案的 Embedding 检索 → 召回更相关的文档
```

原理：答案与答案的语义空间比问题与答案的语义空间更近，从而提升召回精度。

3. Multi-Query（多查询检索）

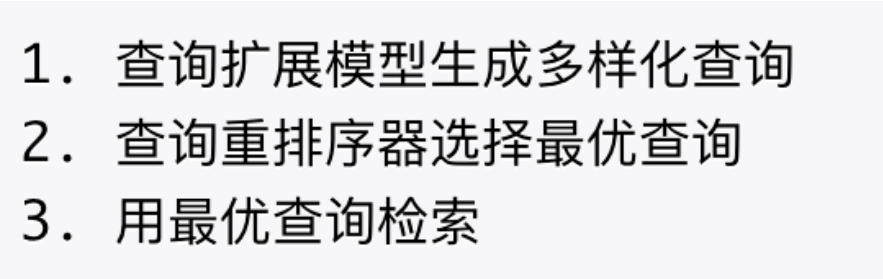
生成多个查询变体，分别检索后合并去重：



4. EAR（Expand, Rerank, and Retrieve）

项目	内容
论文	*Expand, Rerank, and Retrieve: Query Reranking for Open-Domain Question Answering*
链接	https://arxiv.org/abs/2305.17080

核心发现：贪婪解码往往选不到最佳查询扩展。EAR 框架：



效果：

- 领域内 Top-5/20 准确率提升 3-8 个点
- 领域外 Top-5/20 准确率提升 5-10 个点

检索策略选型建议

决策问题	条件	推荐策略
Q1: 数据中是否有大量专业术语/实体名称?	是	必须包含稀疏检索（BM25 / SPLADE）
	否	进入 Q2
Q2: 是否需要语义理解能力?	是（同义词、上下位词等）	必须包含稠密检索
	否	单独 BM25 可能足够
Q3: 能否接受混合检索的复杂度?	是	混合检索（BM25 + Dense + RRF）
	否	选择单一方法
Q4: 用户查询质量如何?	质量高、意图明确	直接检索
	质量参差不齐	查询改写
	开放式问题多	HyDE / Multi-Query
Q5: 语料库规模?	小（<10K 文档）	暴力搜索 / Flat 索引
	中（10K-1M）	HNSW 索引
	大（>1M）	IVF + PQ 或层次化检索

实践建议

1. **从混合检索开始**：BM25 + Dense 的组合通常优于任何单一方法
2. **选择合适的 Embedding 模型**：根据语言、文档长度、资源限制选型
3. **调优融合权重**：在验证集上调整 RRF 的 k 或加权系数 α
4. **引入查询增强**：HyDE 和 Multi-Query 对开放式问题效果显著
5. **关注端到端效果**：使用 ICLERB 思路评估，而非仅关注 NDCG@K
6. **索引类型选择**：HNSW 在精度和性能间取得良好平衡

06 如何获得更好的 Rerank?

为什么需要 Rerank?

向量检索（Embedding + ANN Search）是一种**双编码器（Bi-Encoder）** 架构：Query 和 Document 分别独立编码，通过向量相似度匹配。这种方式速度快，但精度有限——因为 Query 和 Document 之间没有直接交互。

Reranker 采用**交叉编码器（Cross-Encoder）** 架构：将 Query 和 Document 拼接后联合编码，能够捕捉更细粒度的语义交互，显著提升排序质量。

架构	输入方式	编码过程	输出	特点
Bi-Encoder	Query 和 Document 分离	分别独立编码 → Vec_q, Vec_d	Similarity Score	速度快，适合大规模召回
Cross-Encoder	Query + Document 拼接	联合编码	Relevance Score	精度高，适合精排

典型流程：向量检索召回 Top-100 → Reranker 精排 → 取 Top-5 送入 LLM

Rerank 模型选型对比

- 主流 Reranker 模型

模型	参数量	上下文长度	多语言	开源	特点
Cohere Rerank v3	未公开	4096	✓ (100+语言)	✗	商用 API，企业级性能
GTE-Multilingual-Reranker-Base	0.3B	8192	✓	✓	阿里开源，轻量高效
Qwen3-Reranker-0.6B	0.6B	32K	✓ (100+语言)	✓	长上下文，代码检索强
Qwen3-Reranker-4B	4B	32K	✓	✓	均衡之选
Qwen3-Reranker-8B	8B	32K	✓	✓	SOTA 性能
BGE-Reranker-v2-M3	0.6B	8192	✓	✓	BAAI 开源，中文优化
Jina-Reranker-v2	0.3B	8192	✓	✓	轻量，多语言

- 选型建议

决策问题	条件	推荐模型
Q1: 是否需要私有化部署?	是	选择开源模型（GTE/Qwen3/BGE）
	否	Cohere Rerank v3（快速上线）
Q2: 主要语言是什么?	中文为主	GTE-Reranker / Qwen3-Reranker
	英文为主	Qwen3-Reranker-4B
	多语言混合	Qwen3-Reranker / Cohere
Q3: 是否涉及代码检索?	是	Qwen3-Reranker（代码检索 81+）
	否	按其他维度选择
Q4: 文档长度?	长文档（>4K tokens）	Qwen3-Reranker（32K 上下文）
	短文档	任意模型均可

Rerank 实践要点

- 召回数量与精排数量的权衡

典型流程：向量检索 Top-K (K=50~~200) → Reranker → Top-N (N=3~~10) → LLM

- K 太小：可能漏掉相关文档
- K 太大：Rerank 延迟增加，Cross-Encoder 是 O(K) 复杂度

经验值：K=50~100 是常见选择，根据延迟要求调整。

- 截断阈值

Reranker 输出的是相关性分数，可以设置阈值过滤低质量结果：

```
reranked_results = reranker.rerank(query, documents)
filtered = [r for r in reranked_results if r.score > 0.5]  # 阈值根据场景调整
```

- 多路召回 + RRF 融合

当使用混合检索（BM25 + Dense）时，可以用 **Reciprocal Rank Fusion (RRF)** 融合排名，再送入 Reranker：

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

其中 $r(d)$是文档 d 在各路召回中的排名， k 通常取 60。

07 总结

构建一个"好"的知识库，需要在多个环节进行系统性优化：

环节	核心目标	关键决策点
评估体系	定义"好"的标准	RAGAS 指标（Faithfulness、Answer Relevance、Context Precision/Recall）
文档切分	平衡粒度与完整性	切分策略（固定/递归/语义/Late Chunking）、chunk_size、overlap
向量召回	高召回率 + 语义理解	稀疏/稠密/混合检索、Embedding 模型选型、查询增强
重排序	高精度排序	Reranker 模型选型、召回数量 K、截断阈值

核心原则：

1. **评估先行**：先建立评估体系，再迭代优化
2. **从简单开始**：递归切分 + 混合检索 + 轻量 Reranker 是稳健的起点
3. **数据驱动**：根据 bad case 分析定位瓶颈环节
4. **端到端思维**：关注最终生成质量，而非单一环节指标

知识库构建没有标准答案，需要根据数据特点、业务场景和资源约束进行权衡取舍。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 8525

留言

写留言

还没有留言，来写首评